MAPD-B distributed processing exam default projects

Latest revision: **13-May-2026**

Contents

1 Distributed Algorithms **2**

2 Analysis of Covid-19 papers **5**

3 Anomaly detection and Predictive maintenance for industrial devices **9**

4 Batch analysis of cosmic rays using Drift Tubes detectors **17**

5 Streaming processing of cosmic rays using Drift Tubes detectors **24**

6 Streaming processing of the QUAX experiment data for the detection of galactic axions **29**

A Using the CloudVeneto cloud storage **34**

B Changelog **37**

1 Distributed Algorithms

Complexity degree: ■■□

1.1 Introduction

Many algorithms in Data Science and Machine Learning aren't inherently designed to leverage the parallelization capabilities offered by distributed systems. Consequently, various alternative approaches have emerged to adapt existing algorithms to MapReduce-like architectures.
Today, frameworks like Spark and Dask provide a range of pre-implemented distributed algorithms, available either in the Spark MLLib API or within the extensive Python ecosystem used by Dask.
For this project, you'll be tasked with implementing and benchmarking alternative versions of common algorithms in either Spark or Dask, without utilizing the related pre-existing functions. The project's focus lies on efficiently implementing algorithms in a distributed system rather than processing experimental or industrial datasets.

1.2 Task

Based on the documentation provided, you're required to implement one of the listed algorithms in the distributed framework of your preference.
The algorithm's performance must be evaluated using standard synthetic datasets, widely available on platforms such as Kaggle or Scikit-learn.

1.2.1 Clustering - k-Means‖

The k-Means method is widely utilized for unsupervised clustering tasks. One common technique for weight initialization in k-Means is known as k-Means++, and it's primarily a sequential algorithm.
A scalable version, referred to as k-Means‖ (parallel k-Means), has been introduced and described in detail in the paper available at https://arxiv.org/abs/1203.6402.
At the heart of k-Means‖ lies the initialization procedure, which is depicted in the following pseudo-code as presented in the aforementioned paper.

Algorithm 2 k-means‖ (k, ℓ) initialization.

1: $\mathcal{C} \leftarrow$ sample a point uniformly at random from X
2: $\psi \leftarrow \phi_X(\mathcal{C})$
3: **for** $O(\log \psi)$ times **do**
4: $\quad \mathcal{C}' \leftarrow$ sample each point $x \in X$ independently with probability $p_x = \frac{\ell \cdot d^2(x, \mathcal{C})}{\phi_X(\mathcal{C})}$
5: $\quad \mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}'$
6: **end for**
7: For $x \in \mathcal{C}$, set w_x to be the number of points in X closer to x than any other point in $\mathcal{C}$
8: Recluster the weighted points in $\mathcal{C}$ into k clusters

To assess and validate the performance of the distributed algorithm, you can utilize any mid- to large-scale and publicly-available datasets from Kaggle or other sources, e.g. one of the following real-world standard datasets provided by Scikit-learn:

- RCV1 dataset (https://scikit-learn.org/stable/datasets/real_world.html#rcv1-dataset)
- kddcup99 dataset (https://scikit-learn.org/stable/datasets/real_world.html#kddcup-99-dataset)

1.2.2 Clustering - Mini-batch k-Means

Another alternative to k-Means++ and k-Means|| is the Mini-batch k-Means algorithm, detailed in this paper: https://dl.acm.org/doi/10.1145/1772690.1772862 (you can access this link through the University of Padova account).
This approach employs small (mini) batches to optimize k-Means clustering instead of relying on a single large-batch optimization procedure. It has been demonstrated to offer faster convergence and can be implemented to scale k-Means with low computation cost on large datasets.
To assess and validate the performance of the distributed algorithm, you can utilize any mid- to large-scale and publicly-available datasets from Kaggle or other sources, e.g. one of the following real-world standard datasets provided by Scikit-learn:

- RCV1 dataset (https://scikit-learn.org/stable/datasets/real_world.html#rcv1-dataset)
- kddcup99 dataset (https://scikit-learn.org/stable/datasets/real_world.html#kddcup-99-dataset)

1.2.3 Dimensionality reduction

Dimensionality reduction can be achieved through matrix decomposition methods like Singular Value Decomposition (SVD). An alternative approach is to utilize QR decomposition, where a matrix A is decomposed into the product of an orthogonal matrix Q and an upper triangular matrix R.
In the scenario of very large datasets, where the number of features is significantly less than the number of rows (resulting in a "tall-and-skinny" matrix), efficient methods for QR decomposition are crucial.
You're tasked with implementing the Direct Tall-and-Skinny QR (TSQR) method proposed in this paper and evaluating its performance: https://arxiv.org/abs/1301.1071.
To assess its performance, you can test the implementation on any dataset with few features but a large number of records, including real-world datasets as provided by Kaggle or Scikit-learn, e.g.:

- California housing dataset (https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset)
- kddcup99 dataset (https://scikit-learn.org/stable/datasets/real_world.html#kddcup-99-dataset)

1.3 Hints/Suggestions

- Begin prototyping the code in a local environment, preferably using a local cluster deployment of Spark or Dask (docker-compose might be used to spawn a master node and one or two workers).

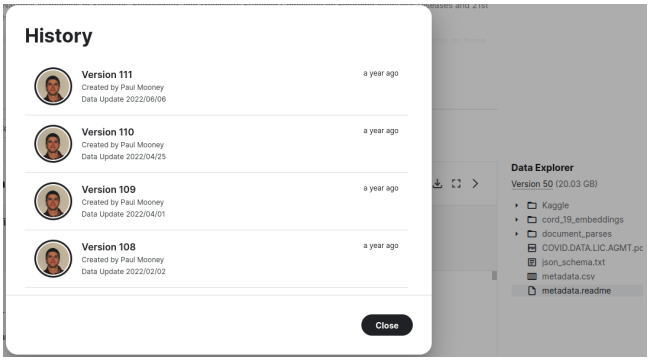
- If the test datasets are large and computationally expensive to process, consider performing a filtering stage to cleanse the dataset from unwanted features. If necessary (only in the case of very large datasets) it’s also possible to reduce the size of the dataset to bring it to a more manageable size.

2 Analysis of Covid-19 papers

Complexity degree: ■□□

2.1 Introduction

This distributed computing project will focus on analyzing papers related to COVID-19, SARS-CoV-2, and other related coronaviruses. The dataset comprises a sub-sample extracted from the original dataset, which consists of over 1,000,000 papers and continues to grow. This dataset is part of the real-world research on COVID-19 known as the COVID-19 Open Research Dataset Challenge (CORD-19). Information about the research and associated challenges can be found on the dedicated page on Kaggle: https://www.kaggle.com/allen-institute-for-ai/CORD-19-research-challenge
You have the flexibility to choose the amount of data to analyze by selecting any version of the dataset you prefer. As of the time of writing, the total dataset size exceeds 80 GB. However, utilizing the Kaggle data explorer functionality, you can opt for an older dataset version (e.g., v50, or older still), as shown in the following screenshot:



2.2 Data Structure

All the documents present in the *data* folder are stored as JSON files with the following structure, taken from the link.
The data may contain empty fields and malformed values, so it's crucial to exercise caution during analysis and processing, and clean/filter data during pre-processing.

```
{
    "paper_id": <str>,                    # 40-character sha1 of the PDF
    "metadata": {
        "title": <str>,
        "authors": [                      # list of author dicts, in order
            {
                "first": <str>,
                "middle": <list of str>,
                "last": <str>,
                "suffix": <str>,
                "affiliation": <dict>,
                "email": <str>
            },
            ...
        ],
        "abstract": [                     # list of paragraphs in the abstract
            {
                "text": <str>,
                "cite_spans": [           # list of character indices of inline citations
                                          # e.g. citation "[7]" occurs at positions 151-154 in "text"
```

```
                                        #      linked to bibliography entry BIBREF3
                    {
                        "start": 151,
                        "end": 154,
                        "text": "[7]",
                        "ref_id": "BIBREF3"
                    },
                    ...
                ],
                "ref_spans": <list of dicts similar to cite_spans>,     # e.g. inline reference to "Table 1"
                "section": "Abstract"
            },
            ...
        ],
        "body_text": [                      # list of paragraphs in full body
                                            # paragraph dicts look the same as above
            {
                "text": <str>,
                "cite_spans": [],
                "ref_spans": [],
                "eq_spans": [],
                "section": "Introduction"
            },
            ...
            {
                ...,
                "section": "Conclusion"
            }
        ],
        "bib_entries": {
            "BIBREF0": {
                "ref_id": <str>,
                "title": <str>,
                "authors": <list of dict>       # same structure as earlier,
                                                # but without `affiliation` or `email`
                "year": <int>,
                "venue": <str>,
                "volume": <str>,
                "issn": <str>,
                "pages": <str>,
                "other_ids": {
                    "DOI": [
                        <str>
                    ]
                }
            },
            "BIBREF1": {},
            ...
            "BIBREF25": {}
        },
        "ref_entries":
            "FIGREF0": {
                "text": <str>,                  # figure caption text
                "type": "figure"
            },
            ...
            "TABREF13": {
                "text": <str>,                  # table caption text
                "type": "table"
            }
        },
        "back_matter": <list of dict>           # same structure as body_text
    }
}
```

2.3 Assignment

2.3.1 Word count distributed algorithm

The first part of the task will see you implement a distributed algorithm to count the occurrences of all words within the list of documents. In Natural Language Processing

(NLP), a document refers to a body of text, where each paper represents a document.
The algorithm is defined as follows:

Map phase : For each document D_i, produce the set of intermediate pairs $(w, cp(w))$, one for each word $w \in D_i$, where $cp(w)$ is the number of occurrences of w in D_i. E.g.: $('hello', 3)$

Reduce phase : For each word w, gather all the previous pairs $(w, cp(w))$ and return the final pair $(w, c(w))$ where $c(w)$ denotes the number of occurrences of w across all documents. In other words, $c(w)$ is equal to $\sum_{k=1}^{n} cp_k(w)$

The algorithm must be executed on the full text of all the papers considered in the dataset. To obtain the full text of a paper, you need to transform the input data by concatenating the strings contained in the `body-text` fields of the JSONs.
We recommend utilizing the RDD/Bag data structure for this transformation. However, if you prefer implementing the algorithm using the DataFrame structure, feel free to do so.
Upon completion of the algorithm, you must identify the most frequently used words, which should show their relevance to viruses and research. For instance, create a barplot of the top words, or perform any other plot to quantify your findings.

2.3.2 Which are the worst and best-represented countries/institutes in the research?

In this part of the project, you'll be required to identify the countries that have been the most and least active in this research field. This analysis should be based on the country of affiliation of the authors. Do the same for the affiliation institute of the authors to identify the universities and research centers that are cited the most/the least.
Potentially, to differentiate from the previous study of Sec. 2.3.1, it's possible (not required) to pass from an RDD/bag to a DataFrame structure for this task.

2.3.3 Obtaining Embeddings for Paper Titles

In Natural Language Processing (NLP), a common technique for text analysis is to transform text into numerical vectors, where each vector represents a word within a document. At the end of the embedding phase, the document is transformed into a list of vectors or a matrix of $n \times m$ where n is the number of words in the document, and m is the size of the vector representing each word n.
For more information about word embedding, refer to this resource.
In this part of the project, you'll have to transform the title of each paper into its embedding version using a pre-trained model. There are several available versions of pre-trained English models. You can choose to use any of the ones proposed at the link, or to opt for the most complete model, available at this other link.
The pre-trained model is essentially a large dictionary in the format `key:vector`.
One of the possible ways to load the model is provided in the official FastText page, and should be re-adapted to be exploited in the context of PySpark/Dask.
Once the model is loaded, you can use the `map` approach to create a DataFrame or a Bag/RDD that is composed of:

- paper-id

- title-embedding

The title embedding can indeed be represented as either a list of vectors or flattened into a single large vector.
When represented as a list of vectors, each vector corresponds to a word in the title, forming a sequence of embeddings that captures the semantic meaning of the title.
Alternatively, when flattened into a large vector, the embeddings of all words in the title are concatenated or aggregated into a single vector representation. This approach reduces the dimensionality of the title embedding while still preserving semantic information, making it suitable for input into machine learning models that require fixed-size input vectors.
Both representations have their advantages and can be chosen based on the specific requirements of the downstream tasks or models being used.

2.3.4 Cosine Similarity Computation

Utilize the vectors generated for each paper title to compute the cosine similarity between each pair of titles.
The cosine similarity measures the similarity between two vectors by computing the cosine of the angle between them, resulting in a value between -1 and 1, where 1 indicates perfect similarity and -1 indicates "perfect dissimilarity".
You will have to compute the cosine similarity between each pair of papers' titles to identify some of the most similar (and most dissimilar) papers in the dataset.

2.4 Hints/Suggestions

- The text of the documents must be sanitized before performing the analysis (ensure that you sanitize the text of the documents *before* performing the various analyses). This typically involves removing punctuation, stop-words, and linking words to focus on meaningful content.
- Keep in mind that not all words in the documents may be present in the pre-trained model. In such cases, simply skip the word and proceed to the next one in the text.
- If you're using a distributed "read file" approach, ensure that all files are located in the same path accessible to each worker. This ensures consistency and prevents errors during data processing.

3 Anomaly detection and Predictive maintenance for industrial devices

Complexity degree: ■■■

3.1 Introduction

In recent years, the combination of IoT (Internet of Things) and Big Data technologies has enabled significant benefits to be derived from data collected by device sensors. While in academic settings this often translates to opportunities for time series forecasting, data prediction, and classification, enterprises focus more on leveraging the vast amounts of data generated by devices for anomaly detection (AD) and predictive maintenance (PM).

- Anomaly Detection involves identifying rare items, events, or observations that deviate significantly from the majority of the data (Wikipedia Anomaly Detection).
- Predictive Maintenance entails determining the condition of in-service equipment to plan when maintenance should be performed and prevent abrupt failures (Wikipedia Predictive Maintenance)

These techniques are crucial for companies to identify potential problems with their devices, detect anomalies during production processes, conduct maintenance proactively to prevent device failures, and reduce the costs associated with unnecessary maintenance.
The following project utilizes real data collected from sensors deployed by a large company operating in the refrigeration field.

3.2 Data Structure

Table 2 provides an example of the dataset that will be used for the analysis. This dataset, extracted from a larger dataset of real-life company equipment collected over the span of years, contains data from four different devices. The columns in the dataset are:

- **when**: is the UNIX timestamp indicating the measurement time of the data
- **hwid**: device identifier for the measured metric
- **metric**: name of the measured metric
- **value**: value of the measured metric

All data have a sampling frequency ranging from 30 seconds to 1 minute, which needs to be normalized to the same frequency before conducting the analysis. The dataset covers the period from October 1, 2020, to March 31, 2021. If cluster resources are limited, the data can be restricted to a three-month period (January 1st to March 31st).
The data are available to this **link** in a zipped csv format. The unzipped version of the dataset is around 5GB.

Table 3 provides details about each variable in the dataset, including its name, physical meaning, measurement unit type, and number of decimals.
Additionally, for variables A5 and A9, which represent alarms on different circuits of a device, they are stored as 16-bit integers in little-endian format but must be converted into 16-bit strings before being used (see Section 3.3.3). This conversion is a typical approach in industrial edge computing.

Suppose there are 16 different sensors on the same device, with each sensor monitoring a specific part of the device.
Each sensor may send a value of 0 if everything is functioning correctly or 1 if their monitored piece of the device is faulty, every N seconds.
However, this individual reporting is not optimal for edge devices that require optimization. Instead, the idea is to combine the outputs of all these sensors into a 16-bit string, where the index of each bit represents the output of the corresponding sensor.

At a certain time T, suppose your 16 sensors send the data as shown in Table 1. This output can be aggregated into a single string: `0100000000000101`, which can then be sent to the final data storage along with the corresponding integer: `8197`.

Table 1: Alarm sensors outputs at certain time T

Sensor	**Output**
Sensor 1	1
Sensor 2	0
Sensor 3	1
Sensor 4	0
Sensor 5	0
Sensor 6	0
Sensor 7	0
Sensor 8	0
Sensor 9	0
Sensor 10	0
Sensor 11	0
Sensor 12	0
Sensor 13	0
Sensor 14	0
Sensor 15	0
Sensor 16	1

Table 2: Example of data taken from the dataset.

When	Hwid	Metric	Value
1601510485159	SW-065	SA4	0
1601510485159	SW-065	SA3	0
1601510485159	SW-065	SA2	0
1601510485159	SW-065	S34	0
1601510485159	SW-065	S33	1
1601510485159	SW-065	S25	0
1601510485159	SW-065	S19	0
1601510485159	SW-065	S17	0
1601510485159	SW-065	S16	0
1601510485159	SW-065	S15	0
1601510532889	SW-115	S41	153
1601510535401	SW-115	S143	100
1601510563238	SW-115	S41	152
1601510565967	SW-115	S143	100
1601510593606	SW-115	S41	153
1601510596120	SW-115	S143	100
1601510624047	SW-115	S41	152
1601510626780	SW-115	S143	100
1601510654443	SW-115	S41	152
1601510657046	SW-115	S143	100

Table 3: Variable description and overview

Name	Description	UDM	Decimals
A5	Circuit 1 Alarm 1		0
A9	Circuit 2 Alarm 1		0
ComError			
E1	Active power		1
E2	Reactive power		1
E3	Active energy (LOW)		0
E4	Active energy (HIGH)		0
E5	Reactive energy (LOW)		
E6	Reactive energy (HIGH)		
P1	Status Control		0
P2	Mode Control		0
P5	Unit Set-Point	°C	1
P6	Set-Point ECO Cooling	°C	1
P7	Set-Point ECO Heating	°C	1
P8	2° Set-Point Cooling	°C	1
P9	2° Set-Point Heating	°C	1
P10	Recovery Set-Point	°C	1
P15	Alarm Reset		0
P16	Set-Point Cooling	°C	1

Table 3: Variable description and overview

Name	Description	UDM	Decimals
P17	Set-Point Heating	°C	1
P18	Set-point ACS	°C	1
S1	Unit Mode		0
S2	Unit Status		0
S3	Set-Point Unit	°C	1
S5	Corrent Power Steps		0
S6	Recovery Set-Point	°C	1
S7	Blocked Alarm		0
S8	Warning Alarm		0
S9	Recovery Request	%	0
S10	Utility Request	%	0
S11	ACS Request		0
S15	Source Pump 1		0
S16	Source Pump 2		0
S17	Source Pump 3		0
S19	Source Inverter Signal	%	1
S25	Anti-Freeze Heater		0
S33	Utility Pump 1		0
S34	Utility Pump 2		0
S35	Utility Pump 3		0
S37	Utility Inverter Signal	%	1
S39	Utility Water IN	°C	1
S40	Utility Water OUT	°C	1
S41	External temperature	°C	1
S42	Water Reset	V	1
S43	Demand Limit	V	1
S45	Free-Cooling Water Temp.	°C	1
S46	Recovery Water inlet	°C	1
S47	Recovery Water OUT	°C	1
S49	Source Water IN	°C	1
S50	Source Water OUT	°C	1
S53	Comulative Alarm		0
S54	Indirect Free-cooling Pump 1		0
S55	Recovery Pump 1		0
S56	Indirect Free-cooling Pump 2		0
S57	Recovery Pump 2		0
S63	DHW valve		0
S64	Free-Cooling Valve		0
S69	Free-Cooling Valve Signal	V	1
S70	Recovery Pump Signal	V	1
S71	Set-Point SH PID1 Circ1	°C	1
S72	Set-Point SH PID2 Circ1	°C	1
S73	Compressor ON C1		0
S80	Set-Point SH PID3 Circ 1	°C	1
S81	SH PID1 Circ 1		1

Table 3: Variable description and overview

Name	Description	UDM	Decimals
S83	SH PID2 Circ 1		1
S86	EEV Position PID2 Circ 1	%	0
S90	EEV Position PID3 Circ 1	%	0
S94	EEV Position PID1 Circ 1	%	0
S97	SH PID3 Circ 1		1
S100	Suction Pressure Cooling Mode Circ 1	bar	1
S101	Suction Pressure Heating Mode Circ 1	bar	1
S102	Discharge Pressure Circ 1	bar	1
S106	Source inlet temp. circ 1	°C	1
S107	Liquid Temperature Circ 1	°C	1
S108	Source Temperature OUT Circ 1	°C	1
S109	Discharge Temperature C1,1	°C	1
S110	Discharge Temperature C2,1	°C	1
S112	4 Way Valve Circ 1		0
S113	Chiller Valve Circ 1		0
S114	Recovery Valve Circ 1		0
S115	Empty Coils Valve Circ 1		0
S117	Status Comp 1,1		0
S118	Status Comp 2,1		0
S122	Signal Inverter Fan Circ 1	V	2
S123	Circ 1 mode		0
S124	Subcooling Circ 1	°C	1
S125	Capacity Circ 1	%	0
S126	Pressure Ratio Circ 1		1
S127	Defrost Mode Circ1		0
S128	Set-Point SH PID1 Circ2	°C	1
S129	Set-Point SH PID2 Circ2	°C	1
S130	Compressor ON C2		0
S137	Set-Point SH PID3 Circ 2	°C	1
S138	SH PID1 Circ 2		1
S140	SH PID2 Circ 2		1
S143	EEV Position PID2 Circ 2	%	0
S147	EEV Position PID3 Circ 2	%	0
S151	EEV Position PID1 Circ 2	%	0
S154	Superheat circ 2 (PID3)		1
S157	Suction Pressure Cooling Mode Circ 2	bar	1
S158	Suction Pressure Heating Mode Circ 2	bar	1
S159	Discharge Pressure Circ 2	bar	1
S163	Source Temperature IN Circ 2	°C	1
S164	Liquid Temperature Circ 2	°C	1
S165	Source Temperature OUT Circ 2	°C	1
S166	Discharge Temperature C1,2	°C	1
S167	Discharge Temperature C2,2	°C	1
S169	Status Comp 1,2		0
S170	Status Comp 2,2		0

Table 3: Variable description and overview

Name	Description	UDM	Decimals
S171	4 Way Valve Circ 2		0
S172	Chiller Valve Circ 2		0
S173	Recovery Valve Circ 2		0
S174	Empty Coils Valve Circ 2		0
S175	(not available)		0
S176	(not available)		0
S178	Signal Inverter Fan Circ 2	V	2
S179	Circ 2 mode		0
S180	Subcooling Circ 2	°C	1
S181	Capacity Circ 2	%	0
S183	Defrost Mode Circ2		0
S201	Status Driver Module 1 Circ 1		0
S202	Status Driver Module 2 Circ 1		0
S203	Status Driver Module 3 Circ 1		0
S204	Status Driver Module 1 Circ 2		0
S205	Status Driver Module 2 Circ 2		0
S206	Status Driver Module 3 Circ 2		0
SA1	Alarm 1 (status)		0
SA2	Alarm 2 (status)		0
SA3	Alarm 3 (status)		0
SA4	Alarm 4 (status)		0
SA5	Circuit 1 Alarm 1 (status)		0
SA6	Circuit 1 Alarm 2 (status)		0
SA7	Circuit 1 Alarm 3 (status)		0
SA8	Circuit 1 Alarm 4 (status)		0
SA9	Circuit 2 Alarm 1 (status)		0
SA10	Circuit 2 Alarm 2 (status)		0
SA11	Circuit 2 Alarm 3 (status)		0
SA12	Circuit 2 Alarm 4 (status)		0

3.3 Assignment

3.3.1 Anomaly Detection

Each device in the dataset contains four different engines responsible for compressing gas to either chill or heat the environment. Continuously turning these engines ON and OFF at a high frequency is not recommended and could indicate problems during installation, deterioration of mechanical parts, or unsuitable external environmental conditions.
For this task, two objectives need to be addressed:

- Distributed Analysis for Anomaly Detection: Perform distributed analysis to identify possible anomalies related to the high frequency of engine turn-on and turn-off events. An anomaly is considered to occur even if just one of the engines of a device is turning on or off with a high frequency.

- Correlation Analysis: Identify possible correlations between the higher frequency of engine turn-on and turn-off events and other variables in the dataset. Are there any patterns or relationships that can be discerned?

By achieving these objectives, potential anomalies related to the high frequency of engine operations can be detected, and insights into correlations with other variables can be gained. This information can help diagnose underlying issues or identify areas for optimization in device operation.
The metrics/variables that contain information about the turning on/off of the engines are:

- S117 (engine 1)
- S118 (engine 2)
- S169 (engine 3)
- S170 (engine 4)

3.3.2 Anomaly Detection 2

As discussed above, each device in the dataset contains four different engines. The percentage of working/loading of these units is crucial as it can indicate issues related to under-dimensioned or over-dimensioned devices, deterioration of mechanical parts, or unsuitable external environmental conditions.
For this task, the objective is to identify possible correlations between the percentage of device loading and the external temperature.
The metrics/variables that contain information about the loading percentage of the devices are:

- S125 (capacity of circuit 1)
- S181 (capacity of circuit 2)

A device can have one or both of these variables. These variables will be used to analyze the loading percentage of the devices and investigate any correlations with external temperature (S41).

3.3.3 Predictive Maintenace

As mentioned in Section 3.2, variables A5 and A9 represent alarms on different circuits of a device.
Specifically, if one or more of the bits in positions 6, 7, and 8 of either or both alarms are equal to 1, it indicates a fault or faults. If at least one of these bits is set to 1, it signifies that the engines are overheating.

For this task, the objectives are:

- Conversion of Alarms: Convert the alarms from their integer encoding to their bit string representation, then correctly identify the required alarms.

- Correlation Analysis: Once the alarms have been identified, investigate possible correlations between the variables and these alarms. Determine if certain variables are associated with specific alarm conditions.
- Predictive Modeling: Utilize the identified correlations to predict the occurrence of alarms in the future. For instance, given a set of correlated features (variables) at time T_{i-1}, attempt to predict whether there will be a fault at time T_i. For this part, it is not required to explore complex algorithms (such as LSTMs or similar). The task is arduous as the nature of the data is far from ideal, being a real-life dataset.

3.4 Hints/Suggestions

- When you need to resample a variable to a different frequency (e.g., from 30 seconds to 1 minute), consider the nature of the variable itself when performing aggregation. For instance, if the variable is an integer representing on/off states or discrete values, using the mean for resampling might not be appropriate. Instead, consider using aggregation methods like minimum or maximum values, which better capture the nature of the data.

4 Batch analysis of cosmic rays using Drift Tubes detectors

Complexity degree: ■■□ This project involves analyzing real data from a particle physics detector to reconstruct the trajectories of muons produced by cosmic rays.

4.1 Introduction

A set of muon detectors, called mini Drift Tubes (miniDTs), has been installed at the Legnaro INFN Laboratories (∼10 km from Padova) to collect signals from cosmic rays. These detectors operate on the principle of ionization: when charged particles (like muons) pass through the detector, they ionize the gas inside, creating electron-ion pairs. An electric field causes the electrons to drift toward an anode wire at a constant velocity ($v_{drift} = 53.8\,\mu m/ns$). Near the wire, the electric field intensifies, amplifying the signal, which is finally collected by the anodic wire (see Figure 1).
The signals are digitized by a Data Acquisition (DAQ) system, where Field Programmable Gate Arrays (FPGAs) perform Time-to-Digital Conversion (TDC). Each digitized signal is called a "hit," representing an ionization event.

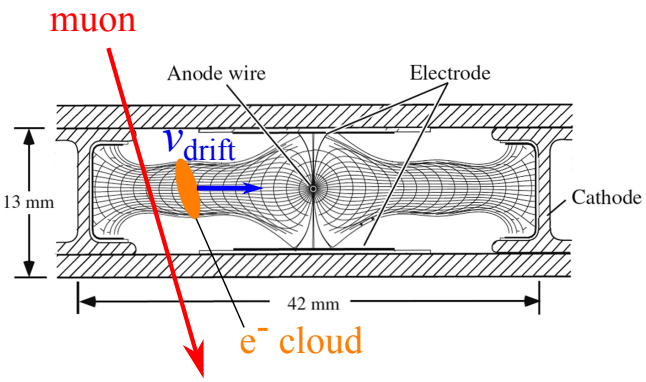


Figure 1: The sensing principle of one Drift Tube cell.

If the time of passage of the muon (usually referred to as t_0) is known, the TDC of a hit can be translated, thanks to the constant v_{drift}, to the position inside the cell where the muon passed, i.e.:

$$x_{hit} = (t_{hit} - t_0)v_{drift}$$

However, there is an intrinsic left-right ambiguity associated with the time-to-space conversion process. This means that the time information of a single hit is insufficient to determine whether the muon passed through the right or left half of the cell.
To resolve this ambiguity the information of multiple cells is required. By analyzing hits from multiple cells, it becomes possible to identify the full track trajectory and resolve the left-right ambiguity.

The geometry of one single detector (one miniDT "chamber") is composed of 64 cells arranged in 4 layers of 16 cells each. Adjacent layers are staggered by half of a cell width. Each cell has a transversal dimension of $42 \times 13\,mm^2$ (width×height), while its length is approximately $60\,cm$. A sketch of the geometry for one DT chamber is reported in Figure 2, where the peculiar cell numbering scheme is also highlighted:

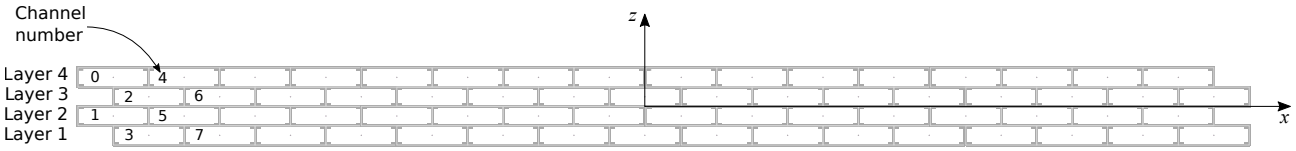


Figure 2: A sketch of the geometry of one DT chamber, comprised of four layers of 16 cells each. Adjacent layers are staggered by half a cell, to improve tracking performance. A local reference frame for each chamber, centered in the barycenter of the mini-DT, sees the z axis vertically pointing to the zenith, the y axis being the direction of the wires, and the x axis being horizontally aligned along the plane across the two innermost layers.

The complete geometry of the entire detector is finally comprised of four DT chambers, stacked on top of each other in a "telescope" configuration. Three chambers (numbered 0, 2, 3) are oriented in the same direction, while the fourth (chamber 1) is rotated 90 degrees, to provide a measure from an orthogonal view. See Figure 3 for a simplified view of the telescope.

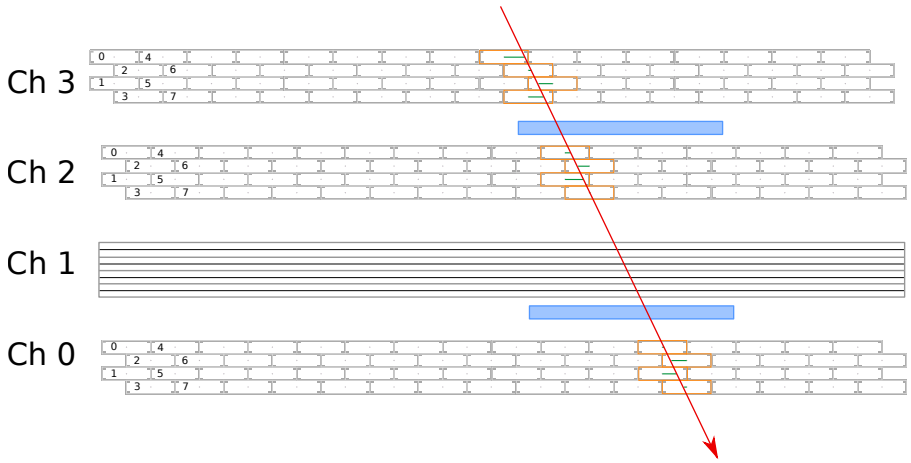


Figure 3: Simplified sketch of the muon telescope configuration. Chambers 0, 2, and 3 provide readings on the same plane, while chamber 1 is installed 90 degrees, to complement the track reconstruction with an orthogonal view.

The positioning of a global reference frame of the 4 chambers has the $z = 0$ coordinate measured on the floor of the experimental hall where the detectors are located. This leads to the following shift in the local (per each chamber) vs. global reference frames:

```
# shift of the reference frame for each Chamber (x, y, z)
shift_chamber = {
  0: {x: 0, y: 0, z:  219.8}, # Ch 0
  1: {x: 0, y: 0, z:  977.3}, # Ch 1
  2: {x: 0, y: 0, z: 1035.6}, # Ch 2
  3: {x: 0, y: 0, z: 1819.8}, # Ch 3
  }
```

This shift is basically the distance from the floor to the bottom of the chamber.
It is important to mention that the time of passage of the muon is typically not known a priori and usually needs to be inferred from external information or geometrical considerations.

In this specific muon telescope configuration, two external detectors (plastic scintillator palettes, shown in blue in Figure 3) provide the external timing information for the passage of particles, hence supplying the t_0 information.

4.2 Dataset and Data structure

The raw dataset is provided on a cloud storage `s3` bucket hosted on Cloud Veneto.

The name of the Bucket is `mapd-minidt-batch`.
The following link can be used to visualize the list all contained files: `https://cloud-areapd.pd.infn.it:5210/swift/v1/AUTH_d2e941ce4b324467b6b3d467a923a9bc/mapd-minidt-batch/`

Instructions on how to use the CloudVeneto Object Storage are available on the CloudVeneto user guide, and in the appendix of this document: `https://userguide.cloudveneto.it/en/latest/ManagingStorage.html#object-storage`.

The dataset is composed of several binary files encoding the stream of hits collected:

- Every hit is encoded as a 64bit word
- Each word has the exact same data structure as all other hits in the file

The 64bit data-format complies to the following standard:

Bit range	Data content
0-4 (5 bits)	`TDC`
5-16 (12 bits)	`BX`
17-48 (32 bits)	`ORBIT`
49-57 (9 bits)	`CHAN`
58-60 (3 bits)	`FPGA`
61-63 (3 bits)	`HEAD`

The data should therefore be “unpacked” from raw files into a structured format.

4.2.1 Dataset cleansing

The `HEAD` information in the data is a header informing which word is associated with TDC hits (marked with `HEAD == 2`) with respect to ancillary data (whose `HEAD` can be 0,1,3,4,5) not used in the context of this project.

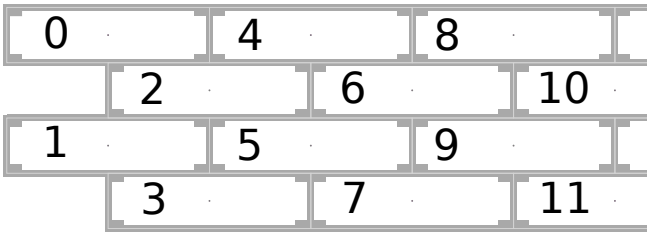
All entries with `(HEAD != 2)` must therefore be dropped from the dataset before performing any further analysis.

4.2.2 Detector and Channel mapping

The mapping between the data format and the chamber and cell geometry is made via a combination of the `CHANNEL` and `FPGA` information:

Chamber number	FPGA value	CHANNEL range
0	0	[0-63]
1	0	[64-127]
2	1	[0-63]
3	1	[64-127]

Within each Chamber, the channel numbering follows a specific recurrent pattern ($+64$ for Chambers 1 and 3):



4.2.3 Hit time information

The digitized time information (TDC) for each signal is provided as a set of 3 counters, mimicking the clock distribution of the LHC collider:

- 1 ORBIT = 3564 BX
  is the period at which one proton bunch circulates around the LHC
- 1 BX = 25 ns
  is the period at which one proton-proton collision (Bunch crossing) may occur at the LHC
- 1 TDC = 1/30 BX
  is the finer time period at which the hit collection is sampled, with a resolution of $\approx 830\,ps$

An "absolute" time (from the start of a data-taking run) can be associated with each hit by converting the counters as follows:

$$t_{\text{TDC hit}}(\text{in ns}) = 25 * (\texttt{ORBIT} * 3564 + \texttt{BX} + \texttt{TDC}/30)$$

The passage time of any muon, t_0, is provided by the external scintillator signal, which is encoded as the TDC hits corresponding to the selection (as an equivalent "extra" mini-DT channel):

- `(FPGA == 1) AND (CHANNEL == 128)`

It must be noted that, due to the delay induced by the signal transmission, the t_0 signal is delayed by roughly 95 ns with respect to the hits collected in the chambers.
This means that the time information from the scintillator will be roughly +95 ns with respect to the actual time of passage of a muon, and must be corrected when comparing the t_0 with the hit $t_{\text{TDC hit}}$.
Dedicated delays can be applied to fine-tune the correction accounting for even finer chamber-by-chamber differences:

```
# scintillator time offset by Chamber
time_offset_by_chamber = {
  0: 95.0 - 1.1, # Ch 0
  1: 95.0 + 6.4, # Ch 1
  2: 95.0 + 0.5, # Ch 2
  3: 95.0 - 2.6, # Ch 3
  }
```

All these adjustments have to be carried out in a pre-processing phase, where the dataset is supposed to get cleaned from unnecessary information, and enriched with the new features required to finally perform the track reconstruction.

4.3 Assignment

Starting from the raw binary data collected by the DAQ of the detector, this project will require reconstructing the local and global tracks, relying on the external timing reference provided by the scintillator.

4.3.1 Pre-selection

Hits from a muon passage are related to each other, being all produced within a maximum drift time of roughly $390\,ns$ (the time an electron cloud would take to drift from the border of the cell to the anodic wire). Genuine muon hits are further supposed to be produced in the same time window as the one provided by the external scintillator t_0.

However, several background components (from electrons to delta rays, including mere electronic noise) can generate additional noise signals.
In Figure 4 it's provided a sketch of the hits (vertical arrows) over time. Noise hits (in magenta) can pollute the data stream, while genuine hits (in black) produced by the muon passage (orange box) are clustered in time, and synchronized with the external scintillator signal (orange arrow, here not corrected for its delay).



Figure 4: Exemplified "time-view" of the data stream.

To reconstruct the muon trajectory inside the detector, it is advised to perform a basic pre-selection:

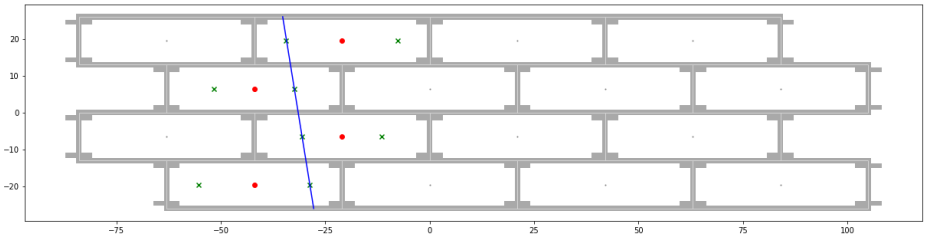
1. Hits must be grouped by Orbit
2. Only Orbits for which a scintillator hit is found in the same orbit should be retained. While multiple muons can cross the telescope in a given orbit, one can opt for a simpler implementation, limiting to those orbits where exactly 1 scintillator hit is found.
3. Each hit should be "placed" in the detector geometry by converting the hit time information into the spatial ones. The nominal drift velocity is $v_{drift} = 53.8\,\frac{\mu m}{ns}$. At this stage, the left/right ambiguity is not resolved, so 2 hit X positions (e.g. `XLEFT`, `XRIGHT`) should be assigned to each hit at this stage.

4.3.2 Track reconstruction

With the dataset correctly preprocessed, it is possible to estimate the trajectory of the muons inside the telescope. With no magnetic field to bend the passage of the particle, the path of the muons is expected to be a straight line.

Local tracks For each chamber, a linear fit should be performed to select the best combination of left/right hits and extract the parameters (angular parameter and intercept) of the "local-track".
A least-squares fit of the hits residuals can be sufficient for this purpose, and the combination with the least $\chi^2/ndof$ can be selected as the one with the highest quality.
(Alternative and smarter approaches can be attempted to speed up the fit)

It could be very useful at this stage to produce plots of the fitted local track and compare it to the location of the hits inside the detector geometry, to check the result of the fit on a small selection of orbits, e.g.:



Global tracks Once the local-track selection is performed for each orbit and Chamber, the hits are unequivocally placed in the detector geometry, and a "global-track" fit can be performed using only chambers 0,2,3 (the ones measuring the same view).
Once more, a least-squares fit can be sufficient, although alternative approaches can be used.

The difference between the angle (in milli-radians) of the global track and the local track measured from Chamber 2 should be measured, as this provides an estimate of the single-detector angular resolution.

4.4 Hints/Suggestions

- Orbits with a very large number of hits are enormously tricky to handle (multiple muons might have crossed the detector or a burst of noise can have mimicked a high detector occupancy). To simplify the task, consider dropping the orbits if the number of hits exceeds a given amount (e.g., 25, but you may investigate which number can be considered reasonable given the task).
- Try to avoid, as much as possible, performing loops over all the possible left/right combinations in an orbit, but parallelize as much as possible the fitting algorithm.

- For the "unpacking" of the raw data, the use of RDDs/Bags is typically the most suitable. Once the data is interpreted as a structured dataset, it might be useful (depending on the processing you have in mind) to move to Dask/pySpark DataFrames.

5 Streaming processing of cosmic rays using Drift Tubes detectors

Complexity degree: ■■■ The goal of this project is to reproduce a real-time processing of real data collected in a particle physics detector and publish the results in a dashboard for live monitoring.

5.1 Introduction

The introductory set of information on the underlying physics and detector details is shared with Project 4 and can be found in Section 4.1 of this document.

5.2 Data strucure

The raw dataset is provided on a cloud storage `s3` bucket hosted on Cloud Veneto.

The name of the Bucket is `mapd-minidt-stream`.
The following link can be used to visualize the list all contained files: `https://cloud-areapd.pd.infn.it:5210/swift/v1/AUTH_d2e941ce4b324467b6b3d467a923a9bc/mapd-minidt-stream/`

Instructions on how to use the CloudVeneto Object Storage are available on the CloudVeneto user guide, and in the appendix of this document: `https://userguide.cloudveneto.it/en/latest/ManagingStorage.html#object-storage`.

The dataset is composed of multiple `txt` files formatted as comma-separated values, where each record (row) can be associated with each new signal processed in the DAQ of the detector.

The attributes' names are the following:

- `TDC_MEAS`
- `BX_COUNTER`
- `ORBIT_CNT`
- `TDC_CHANNEL`
- `FPGA`
- `HEAD`

A simple inspection of the data will provide something like the following:

	HEAD	FPGA	TDC_CHANNEL	ORBIT_CNT	BX_COUNTER	TDC_MEAS
0	1	1	0	3387315431	0	130.000000
1	0	1	2	3387315431	1119	24.000000
2	4	1	0	3387315431	0	-0.573730
3	5	1	0	3387315431	0	45.500000
4	2	0	75	3387200947	2922	2.000000
5	2	0	105	3387200955	2227	29.000000

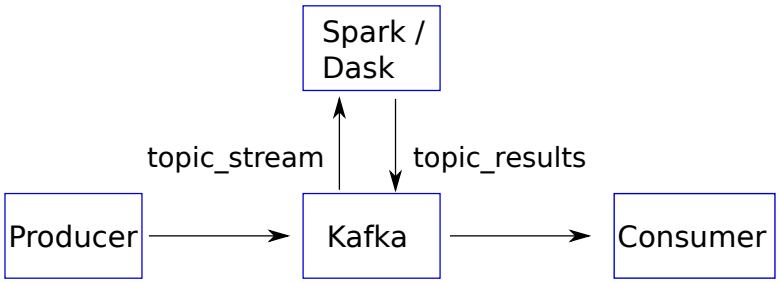
6	2	0	107	3387200955	2234	7.000000
7	2	0	126	3387200973	476	29.000000
8	2	1	55	3387200955	1797	12.000000
9	2	1	36	3387200956	2165	28.000000
10	2	1	51	3387200970	249	14.000000
11	2	1	90	3387200973	475	28.000000
12	4	0	0	3387339329	0	-0.537109
13	5	0	0	3387339329	0	-41.531250
14	3	0	0	3387339394	4353	0.000000

5.3 Task

This project involves creating a streaming application to monitor basic detector quality plots in real-time, using data collected by the DAQ of the detector. This activity is common for all real-life detectors, where the continuous monitoring of the data collected (Data Quality Monitoring, or DQM) is vital for a successful data collection campaign.

Data should be produced to a Kafka topic by a stream-emulator script that reads files in small (micro) batches.
The processing of the hits will then be performed in a distributed framework such as Dask or pySpark.
The processed results will need to be re-injected into a new Kafka topic hosted on the same brokers.
A final consumer, ideally even a simple Python script, will handle the live plotting and display updates of the evaluated quantities online.
(`bokeh` or `matplotlib`'s `pyplot` are good choices for creating simple "dashboards", but other solutions can also be applied for this task, for example `streamlit`).

The overall schema of the architecture for this project can be simplified as follows:



5.3.1 Data streaming

Data should be injected into a Kafka topic by a Python script emulating a continuous DAQ stream.

A realistic data production rate is about 1000 records/sec (i.e. 1000 rows/sec, or 1kHz of DT hits rate). However, it might be beneficial to create a tunable-rate producer to adjust the emulated hit rate.

Note that reading events from disk, if not carefully handled, might result in an effective production rate different from the chosen one.

5.3.2 Dataset processing

The processing can be performed using either Spark or Dask.
It is recommended to work with batches of data lasting a few seconds (1 to 5 seconds may be a good starting point, but the optimal batch time is specific to each application). It's important to remember that the shorter the batch size, the faster the processing should be to keep up with data polling.
However, longer batch sizes result in larger datasets to be processed for each batch.

The data processing should start with a data-cleansing step where all entries with `HEAD != 2` are dropped from the dataset, as they provide ancillary information not relevant to the project.

The mapping between the data format and the detectors is as follows:

Chamber number	`FPGA`	`TDC_CHANNEL` range
0	0	[0-63]
1	0	[64-127]
2	1	[0-63]
3	1	[64-127]

The following information should be produced for each batch:

1. Total number of processed hits after cleansing: 1 scalar value per batch.
2. Total number of processed hits (after cleansing), by chamber: 4 scalar values per batch.
3. Histogram of the counts of active `TDC_CHANNEL` per chamber: 4 arrays per batch.
4. Histogram of the total number of active `TDC_CHANNEL` in each `ORBIT_CNT`, per chamber: 4 arrays per batch.

These pieces of information should be wrapped in one message per batch and injected into a new Kafka topic.

5.3.3 Live Plotting

The results of the processing should be presented in the form of a continuously updating dashboard.
The DQM script (in Python or any other language) should consume data from the topic and create plots and histograms of the aforementioned quantities.
Simple Python modules such as `Bokeh` or `pyplot` (in its free version) can help create a continuously updating webpage for visualizing the plots. However, any other solution may be considered viable for this part of the task.

5.3.4 Extras

Provided the main goal of the project is met, additional tasks can be carried out on this dataset by extending the distributed processing studies to produce additional features.

A piece of external timing information, measuring the exact time of passage of a muon, is provided in the telescope by plastic scintillator palettes (once again, refer to the details of Project 4, in Section 4.1).
By selecting only the hits compatible with a scintillator signal, we can observe interesting features in the data. For instance, a 'muon-radiography' can be performed, showing only the number of active channels compatible with the passage of the muons.
Additionally, the properties of the detector impose an upper bound on the hit collection time: the electron cloud produced by any given muon cannot travel further than half of a cell before being collected on the sensing wire.
Plotting the difference between the hit collection times and the time of passage of the related muon, the resulting distribution is expected to exhibit a box-shaped profile with sharp rising and falling edges.

To be able to define these quantities, the hit time information should be translated into an absolute timing feature.
The TDC time information for each signal is provided as a set of 3 counters, mimicking the clock distribution of the LHC collider:

- 1 `ORBIT_CNT` = 3564 `BX_COUNTER`
  is the period at which one proton bunch circulates around the LHC
- 1 `BX_COUNTER` = 25 ns
  is the period at which one proton-proton collision may occur at the LHC
- 1 `TDC_MEAS` = 1/30 `BX_COUNTER`
  is the finer time period at which the hit collection is sampled, with a resolution of $\approx 830\,ps$

An "absolute" time (from the start of a data-taking run) can be associated with each hit by converting the counters as follows:

$$t_{\text{TDC hit}}(\text{in ns}) = 25 * (\texttt{ORBIT_CNT} * 3564 + \texttt{BX_COUNTER} + \texttt{TDC_MEAS}/30)$$

The passage time of any muon, t_0, is provided by the external scintillator signal, which is encoded as the TDC hits corresponding to the selection (as an equivalent "extra" mini-DT channel):

- `(FPGA == 1) AND (CHANNEL == 128)`

It must be noted that, due to the delay induced by the signal transmission, the t_0 signal is delayed by roughly 95 ns with respect to the hits collected in the chambers.
This means that the time information from the scintillator will be roughly +95 ns with respect to the actual time of passage of a muon, and must be corrected when comparing the t_0 with the hit $t_{\text{TDC hit}}$.
Dedicated delays can be applied to fine-tune the correction accounting for even finer chamber-by-chamber differences:

```
# scintillator time offset by Chamber
time_offset_by_chamber = {
  0: 95.0 - 1.1, # Ch 0
  1: 95.0 + 6.4, # Ch 1
  2: 95.0 + 0.5, # Ch 2
  3: 95.0 - 2.6, # Ch 3
  }
```

An ABSOLUTETIME information should thus be associated with all hits during the processing phase by converting the counters to the proper time information in ns.
Only for those hits with a scintillator signal within the same orbit, a DRIFTIME can be finally defined, corresponding to the ABSOLUTETIME difference between each hit and the scintillator (from the same orbit).
In case of multiple scintillator hits in the same orbit, arbitrarily choose one (e.g. the min one).
Two additional types of results can be added to the previous list, and displayed on the live visualization:

1. histogram of the counts of active TDC_CHANNEL, per chamber, ONLY for those orbits with at least one scintillator signal in it: 4 arrays per batch

2. histogram of the DRIFTIME, per chamber: 4 arrays per batch

5.4 Hints/Suggestions

- Start by prototyping the processing for a single static batch before extending to the streaming part.

- When dealing with structured datasets, consider using Dask or pySpark DataFrames directly instead of Bags or RDDs.

- The Kafka messages resulting from distributed processing could be formatted to encode, for each histogram, an array of bins and counts. For instance, a JSON structure like the following might be one (although not the only one!) viable alternative:

```
msg_json = {
  "histo_1": { "bin_edges": [...],
               "bin_counts": [...]
             },
  "histo_2": {...},
  ...
}
```

6 Streaming processing of the QUAX experiment data for the detection of galactic axions

Complexity degree: ■■■ The goal of this project is to establish a quasi real-time processing chain for the data generated by the QUAX experimental apparatus. Additionally, the aim is to develop a live monitoring system for the detector data.

6.1 Introduction

The axion is a hypothetical particle proposed to address the strong CP problem of Quantum Chromo Dynamics.

Additionally, there is speculation that axions may constitute a portion of the dark matter content in our Galaxy. Astrophysical observations and cosmological considerations suggest a favored mass range of $1\,\mu\mathrm{eV} < m_a < 10\,\mathrm{meV}$.

The QUAX experiment aims to detect axions by utilizing a copper cavity immersed in a static magnetic field of $8.1\,\mathrm{T}$, cooled down to a working temperature of approximately $150\,\mathrm{mK}$. Axions are expected to couple with a photon and induce a radio-frequency signal that can be detected using a Josephson parametric amplifier.

For a given configuration of the RF cavity, a frequency scan of the electromagnetic field is conducted to potentially identify a localized excess, indicating the coupling of an axion with the photon.

Other experiments may use the coupling of the axion with the spin of fermionic particles as a detection mechanism. However, this second detection method is not discussed here, since the dataset of this project relates to the first method.
The data acquisition system of the QUAX experiment generates two streams of digitized reading of the amplifiers, representing the amplitudes of two amplitude-modulated sinusoids that are in quadrature phase (more information on IQ on Wikipedia).

To enhance the signal-to-noise ratio, a QUAX data-taking run spans a long duration, extending up to weeks. During this time, scans are repeated multiple times.
Data collected during these runs are saved locally on the DAQ servers as binary files. Each binary file corresponds to a series of continuous scans conducted across the entire frequency range. Therefore, a single pair of raw files represents only a few seconds of data collection, yet encompasses several thousand scans.
This approach allows for comprehensive data collection while mitigating the dominance of noise in individual scans.

The processing of the raw data consists of two phases:

1. For each scan, perform a Fourier transform to transition from the time domain to the frequency domain.

2. Average all scans within a data-taking run, grouping them into frequency bins, to derive a single frequency scan

This relatively simple procedure is highly parallelizable and should be implemented in a quasi-online pipeline for two main reasons:

- Monitoring the scans during data collection is crucial to promptly identify possible issues in the detector setup or instabilities in the experiment's conditions. Running this procedure on-the-fly and visualizing the results in a live-monitoring dashboard can provide valuable information and prevent wasting weeks-long runs.
- Data is continuously produced at a very high rate, and the local storage provided by the DAQ server of the QUAX experiment is not suitable for large-volume and long-lasting datasets. Therefore, a real-time ETL (Extract-Transform-Load) pipeline is envisioned to handle the data efficiently.

Implementing this pipeline would ensure that data is processed in real-time, allowing for timely analysis and visualization of results, while also addressing the challenges associated with data storage and management.

6.2 Data strucure

The dataset is provided on a cloud storage s3 bucket hosted on Cloud Veneto.

Name of the Bucket: `quax`
Link to visualize the list all contained files: `https://cloud-areapd.pd.infn.it:5210/swift/v1/AUTH_d2e941ce4b324467b6b3d467a923a9bc/quax/`

Instructions on how to use the CloudVeneto Object Storage: `https://userguide.cloudveneto.it/en/latest/ManagingStorage.html#object-storage`

The dataset is composed of 2 sets (named `duck_i` and `duck_q` for the in-phase and quadrature components, respectively) of `.dat` binary files, each one comprised of a continuous series of ADC readings from the amplifier.
Each ADC reading is written in the raw files as a `32 bit` floating point value in little-endian format. For instance this can be interpreted in plain Python using a `struct.unpack` call.
The ADC readout frequency is $2 \times 10^6\,\mathrm{Hz}$ (2 MegaSample per second, or 2MS/s), thus resulting in a total raw data throughput of 128 Mbps (16 MB/s).
During data taking the readouts are formatted in `.dat` file such that each file is comprised of 2^{23} samples. This results in producing a pair of `.dat` files (`duck_i` and `duck_q`) every $4.2\,\mathrm{s}$.

6.3 Task

Starting from data collected by older runs of the DAQ of the detector, this project will involve creating a streaming application to monitor basic detector quality plots in real-time.

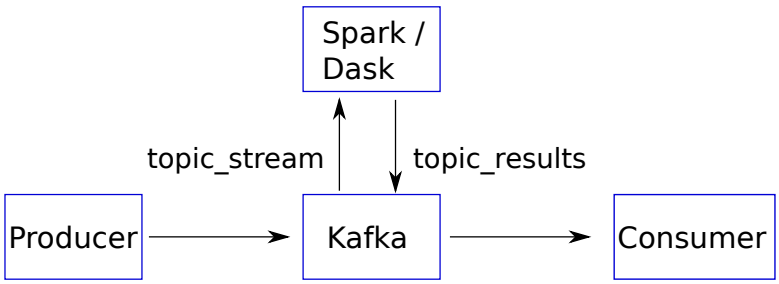
Data will be generated to a Kafka topic by a stream-emulator script, utilizing files from a previous QUAX data-taking run stored in the CloudVeneto object store area. Each file is expected to be produced approximately every few (around 5) seconds, owing to the

fixed ADC scanning rate and the predetermined size of files written to disk. However, it's beneficial to develop a straightforward Python script capable of adjusting the file processing rate as needed.

The processing of each run will be executed in a distributed framework such as Dask (preferably), or pySpark. The processing pipeline will begin with the pair of files containing the two phase components readings (real and imaginary or in-phase and quadrature).
These files will need to be unpacked based on their schema, as discussed above, and divided into scans.
For each scan, a Fast Fourier Transform (FFT) will be performed in parallel. Finally, the results of all FFTs will be merged to generate a single "averaged-FFT".
The results of the processing, namely the "averaged-FFT" for each pair of files, will need to be re-injected into a new Kafka topic hosted on the same brokers.

A final consumer, ideally implemented as a simple Python script, will handle the plotting and display of live updates for the scans evaluated online. This consumer will publish a new plot approximately every 5 seconds (the plotting frequency might be adapted, if necessary) and continuously update the entire "run-wide" scan. Tools such as `bokeh` or `pyplot` are suitable choices for creating simple "dashboards", although other solutions can also be applied for this task.

The overall schema of the architecture for this project can be simplified as follows:



6.3.1 Data streaming

The raw data files should be injected in pairs into a Kafka topic by a process emulating a quasi-continuous DAQ stream of files.
A realistic data production rate is about 1 file-pair every 5 seconds.
However, as previously stated, it might be beneficial to create a tunable-rate producer to adjust the rate, and opt for a slower data ingestion rate in the case of issues with the processing layer.
Data can be transmitted in Kafka messages of either entire pairs of files, or in subsets (slices) of these, still preserving the appropriate throughput.

6.3.2 Data processing

The size of the files produced by the DAQ is 32 MB each.
The data processing must proceed by first reading records in pairs (`q` and `i`), and con-

verting from binary to floating point values.

Both the `q` and `i` components should be sliced according to the 32-bits float data format and the readout frequency. This process results in 2^{23} float values from each file. The `q` and `i` components must be combined together consistently, and a FFT will have to be performed to extract the relative power spectrum.

To perform the FFT it should be used a number of bins equal to $n_{bins} = 2^{11} = 2048$.
This will results in a total of $\frac{2^{23}}{2^{11}} = 2^{12} = 4096$ FFTs performed on each pair of `q` and `i` files.
To perform the Fourier Transform one can have a full bandwidth of $2\,\mathrm{MHz}$, spanning the range $(-1\,\mathrm{MHz}, 1\,\mathrm{MHz})$.
For each pair of files (per batch), an average of all FFTs is calculated to produce an averaged-out power spectrum, effectively reducing the noise contribution.

The output of the processing for each batch should be encapsulated in one message containing all the relevant features to represent the input data. At a minimum, the message should include the power spectra values for each FFT bin and the standard deviation of each bin.

The processed data should then be sent to another Kafka topic for subsequent consumption.

6.3.3 Live Data Monitoring Dashboard

The results of the processing should be presented in the form of a continuously updating dashboard, mimicking live data monitoring of the detector.

At a minimum, one plot should be displayed and updated for each batch, representing the power spectrum of the newly processed batch along with its error.

Additionally, a "cumulative" power spectrum, resulting from the cumulative average of all batches processed during the run, may be included for comprehensive analysis.

A Python script (or any other language) should consume data from the topic and generate the plots.
Simple Python modules such as `Bokeh` or `pyplot` (in its free version) can help create a continuously updating webpage for visualizing the plots. However, you are encouraged to explore and apply any other solution for this part of the task that you might consider viable.

6.4 Hints/Suggestions

- Start by prototyping the processing for a single static batch before extending to the streaming part
- Dealing with an unstructured dataset, it might be useful to initially use Dask or pySpark low-level APIs (Bags or RDDs) for the first data ingestion part of the task

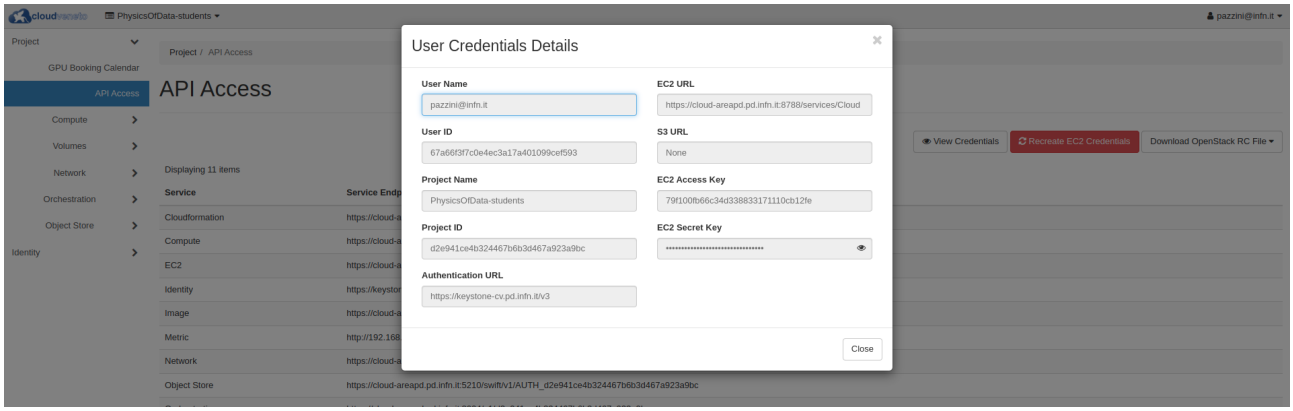
- The Kafka messages resulting from distributed processing could be formatted to encode, for each spectrum, an array of bins and values. For instance, a JSON structure like the following might be one (although not the only one!) viable alternative::

```
msg_json = {
  "average" = { "frequency" = [...],
                "value"= [...],
                "rms"= [...]
              },
}
```

A Using the CloudVeneto cloud storage

In CloudVeneto a Cloud Storage solution is implemented. Users can interact with this object store through the Amazon S3 (Simple Storage Service) API.
Please refer to the CloudVeneto documentation guide before starting to use it.
Most importantly, read the part concerning the access of the storage using the S3 interface, which also describes how to get your credentials, at the link.
To get your own AWS (Amazon Web Services) EC2 (Elastic Compute Cloud) credentials, in the CloudVeneto Dashboard you will have to navigate to `Project`, then `API Access`, and then click on `View Credentials`.

The `aws access key` (public) and the `aws secret key` (private!) can be found as `EC2 Access Key` and as `EC2 Secret Key` respectively.



Accessing a file from Python for testing purposes If we want to download a few files from S3, to use them in a local environment, we can use the `boto3` Python module. The documentation of this module can be found here. The library allows to operate quite transparently with the remote CloudStorage, creating new buckets (equivalent to folders), listing objects (equivalent of files), and downloading as well as ***deleting*** data. Please exercise maximum caution, as maintaining and preserving the content of CloudVeneto object storage is the responsibility of all users. Please note the two warnings that can be found on the official CloudVeneto documentation :

Warning

Please note that backups are not performed on the CloudVeneto object storage.

Warning

Please note that containers (and their objects) are owned by the project, and not by the individual who created them. This means that, if you create a container and upload some files on this container, the other members of the project can see (and also delete) these files.

If interested in accessing a given bucket to download a file, it's possible to follow the code example provided below:

```
import boto3
s3_client = boto3.client('s3',
                         endpoint_url='https://cloud-areapd.pd.infn.it:5210',
                         aws_access_key_id='79f100fb66c34d338833171110cb12fe',
                         aws_secret_access_key='--------------------------------',
                         verify=False)
s3_client.download_file('mapd-minidt-stream', 'data_000001.txt','my_local_file.txt')
```

For simplicity, credentials are directly included in all examples in this section. In real deployments, credentials should preferably be managed through environment variables or configuration files.

This example will download the `data_000001.txt` file (in proper terms, the *object* having `data_000001.txt` as the relative *key*) from the `mapd-minidt-stream` folder (in proper terms, *bucket*), into your local folder under the name `my_local_file.txt`.

Accessing data using Dask Dask relies on the `boto3` and `s3fs` libraries to handle requests to remote object stores via S3, so a very similar implementation to the previous one can be performed. As always, please take a look at the documentation first.
For instance, if willing to load all objects in a bucket into a Dask DataFrame, one could do:

```
import dask.dataframe as dd
df = dd.read_csv('s3://mapd-minidt-stream/data_*.txt',
                 storage_options={
                     'key':'79f100fb66c34d338833171110cb12fe',
                     'secret':'--------------------------------',
                     'client_kwargs': {
                         'endpoint_url': 'https://cloud-areapd.pd.infn.it:5210',
                         'verify':False
                     }
                 })
# print the content of the df
df.head()
```

Accessing data using Spark (v4.1.1) Spark does not rely on `boto3` to handle requests to the remote object store via S3, but instead it relies on a `hadoop`-S3 "connector".
To instruct Spark to handle the requests to the remote object store one should follow a few steps.

1. In each node of the cluster, import the CloudVeneto certificate

```
wget https://raw.githubusercontent.com/CloudVeneto/CertCA/master/CloudVenetoCAs.pem
sudo keytool -trustcacerts \
        -keystore /usr/lib/jvm/java-17-openjdk-amd64/lib/security/cacerts \
        -storepass changeit \
        -alias CloudVeneto \
        -importcert -file CloudVenetoCAs.pem
```

2. Start the Spark master and workers normally

```
cd $SPARK_HOME
./sbin/start-all.sh
```

3. When creating a Spark Session, load the proper java modules.

```
from pyspark.sql import SparkSession

# create a spark session including modules to connect to s3
spark = SparkSession.builder \
    .master("spark://master:7077")\
    .appName("Your Spark Application Name")\
    .config('spark.jars.packages', 'org.apache.hadoop:hadoop-aws:3.4.1,org.apache.hadoop:hadoop-common:3.4.1')\
    .config("spark.sql.execution.arrow.pyspark.enabled", "true")\
    .config("spark.sql.execution.arrow.pyspark.fallback.enabled", "false")\
    .config('spark.hadoop.fs.s3a.aws.credentials.provider', 'org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider')\
    .config('spark.hadoop.fs.s3a.access.key', '79f100fb66c34d338833171110cb12fe')\
    .config('spark.hadoop.fs.s3a.secret.key', '--------------------------------')\
    .config('spark.hadoop.fs.s3a.endpoint', 'https://cloud-areapd.pd.infn.it:5210')\
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem") \
    .config("spark.hadoop.fs.s3a.metadatastore.impl", "org.apache.hadoop.fs.s3a.s3guard.NullMetadataStore") \
    .config("spark.hadoop.fs.s3a.path.style.access", "true") \
    .config("spark.hadoop.fs.s3a.connection.ssl.enabled","false") \
    .config("com.amazonaws.sdk.disableCertChecking","true") \
    .getOrCreate()

# test with a `take` of 3 elements from one file
spark.read.format("text").load('s3a://mapd-minidt-stream/data_000000.txt').take(3)

# extend the test to count all entries out of 10 files
spark.read.format("text").load('s3a://mapd-minidt-stream/data_00001*.txt').count()
```

At the first execution, Spark/Ivy will automatically download the required Hadoop/S3 connector modules from the official central repositories, and a message similar to this will appear:

```
downloading https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-aws/3.4.1/hadoop-aws-3.4.1.jar ...
        [SUCCESSFUL ] org.apache.hadoop#hadoop-aws;3.4.1!hadoop-aws.jar (182ms)
downloading https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-common/3.4.1/hadoop-common-3.4.1.jar ...
        [SUCCESSFUL ] org.apache.hadoop#hadoop-common;3.4.1!hadoop-common.jar (128ms)
downloading https://repo1.maven.org/maven2/software/amazon/awssdk/bundle/2.24.6/bundle-2.24.6.jar ...
        [SUCCESSFUL ] software.amazon.awssdk#bundle;2.24.6!bundle.jar (5038ms)
    ...
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   89  |   89  |   89  |   0   ||   89  |   89  |
        ---------------------------------------------------------------------
```

Later, these packages will already be located locally, and no further downloads will be required:

```
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   89  |   0   |   0   |   0   ||   89  |   1   |
        ---------------------------------------------------------------------
```

B Changelog

15-05-2026
1. Updates to the S3 section, adjusting to PySpark v4.1.1

10-05-2026
1. Updated link for mini-batch k-means algorithm and added a suggestion to use Docker to debug.
2. Uniform section structure.
3. Added suggestions to Project “Streaming processing of cosmic rays using Drift Tubes detectors”

17-05-2025
1. Included new figures, and additional description for the MiniDT batch project
2. Reshaped the s3 appendix to clarify key aspects and prevent disruptions as in 2024

16-05-2025
1. Rewording of most projects
2. Re-uploaded the `minidt-batch` and `minidt-stream` dataset on CloudStorage after last year students’ deletion

01-07-2024
1. Updated the S3 section for PySpark

13-05-2024
1. Cross check and minor mods to the CloudStorage appendix for Python and Dask parts (Spark still to be updated).
2. Review of the QUAX project. Fixups to the previously wrong number of bins for FFT.
3. Review of the MiniDT projects. Mostly revisiting text.
4. Review of the Covid-19 project. Alternative embedding models.
5. Review of the Industrial maintenance project. Mostly textual revisions.

03-05-2024
1. Minor textual review of the Distributed Algorithms project

08-06-2023
1. Minor review and revision of the QUAX data processing project

28-05-2023
1. Included note on how to retrieve a slimmer version of the Kaggle dataset for the Covid-19 project
2. Included a first (almost preliminary) version of the QUAX stream data processing project

30-05-2022
1. Modified the pointer to the S3 buckets of the cloud storage files for both Batch and Streaming projects of Drift Tube detectors
2. Included Appendix on the S3 connection using plain python/Dask/Spark